

Documentation Technique - FruitPower

Table des versions

Version	Date	Auteur	Changements
1.0	2023-11-09	Ruben Wihler	...

Table des matières

▼ Documentation Technique - FruitPower

- [Table des versions](#)
- [Table des matières](#)
- [Introduction](#)

▼ Rappel de l'énoncé

- [Organisation](#)
- [Livrables](#)
- [Matériel et logiciels à disposition](#)

▼ Méthodologie

▼ Méthode en 6 étapes

- [1. S'informer](#)
- [2. Planifier](#)
- [3. Décider](#)
- [4. Réaliser](#)
- [5. Contrôler](#)
- [6. Evaluer](#)

- [Méthode Agile](#)

- [Sauvegardes et versionning](#)

▼ Planification

- [Planning prévisionnel](#)
- [Planning effectif](#)
- [Product backlog](#)

▼ Tâches techniques

- 000 : Préparation du projet
- 001 : Implémentation VR
- 002 : Environnement 3D
- 003 : Main du joueur
- 004 : Arbres et buissons
- 005 : Génération de fruits
- 006 : Ramassage de fruits
- 007 : Disparition des fruits
- 008 : Compteur de points
- 009 : Visualisation des points
- 010 : Compteur de temps
- 011 : Fin de partie
- 012 : Score final
- 013 : Rejouer
- 014 : Musique et bruitages
- 015 : Graphismes
- 016 : Modèles 3D
- 017 : Post-traitement
- 018 : Interface

▼ Analyse organique

- Informations générales

▼ Gestions de la réalité virtuelle

- Mains du joueur
- Déplacement du joueur

▼ Utilisation dans Unity

- XR Plugin Management
- XR Interaction Toolkit

▼ Utilisation dans la scène

- XR Player
- LeftHand et RightHand

- Environnement 3D

▼ Gestion des parties

- Classes du système de gestion de partie
- GameManager
- GameOption
- GameScore
- GameTimer
- GameStats

▼ Gestion des fruits

- Classes du système de fruit
- FruitManager
- FruitPooler
- FruitPoolData
- Fruit
- FruitState
- FruitTypeData
- FruitTypesDatas
- FruitSpawner
- FruitSpawnerManager
- Basket
- Interface utilisateur

▼ Implémentation

▼ Généralités concernant l'implémentation

- Bibliothèques et outils externes
- Analyse des fonctionnalités majeures

▼ Sécurité

- Obfuscation

▼ Plan de test

- Périmètre

▼ Cas de test

- Tests manuels
- Tests automatisés
- Journal de test

▼ Conclusion

- Difficultés rencontrées
- Variantes de solutions et choix
- Améliorations possibles
- Bilan personnel
- Remerciements

▪ Références

▼ Annexe

▼ Utilisation de chatGPT

▼ Convention doublons documentation

- prompt
- réponse
- Code source

Introduction

Ce document est un rapport présentant différents aspects de la conception du projet FruitPower. Ce projet a été réalisé dans le cadre du Travail pratique individuel (TPI) durant la session de mai 2024. Il a pour but de valider mes compétences acquises pendant la formation Informaticien CFC dispensée à l'école d'informatique du CFPT au Petit-Lancy. FruitPower est un jeux vidéo utilisant la réalité virtuelle. Il a été développé en C# avec le moteur de jeu Unity.

Rappel de l'énoncé

Les informations sont extraites du cahier des charges du TPI.

Organisation

Elève	Maître d'apprentissage	Experts
Ruben Wihler	M. J. Aliprendi	Mickaël Strazzeri, Yvan Poulin

Livrables

Pour les experts et le maître d'apprentissage :

- Rapport de projet
- Manuel utilisateur
- Résumé du rapport du TPI
- Journal de travail
- Version compilée et sources du projet Unity C#

Matériel et logiciels à disposition

- Un PC standard école, 2 écrans
- Windows 10
- Visual studio code
- Visual studio 2022
- Unity 2022.3.12f1
- Suite Office

Méthodologie

Méthode en 6 étapes

1. S'informer

Lors de cette étape, j'ai dû m'informer sur le cahier des charges, l'analyser en profondeur pour bien comprendre toutes les fonctionnalités demandées. C'est également durant cette étape que j'ai demandé des questions/informations à M.Aliprandi pour tout ce qui concerne mes différentes incompréhensions. A chaque fois que je commençais une storie je m'informais en regardant le cahier des charges.

2. Planifier

Dans cette étape, j'ai créé un planning prévisionnel pour pouvoir m'organiser et savoir ce que je dois faire et quand. Pour faire ce planning, j'ai dû découper le travail en plusieurs tâches. Pour ces tâches, j'ai décidé de les mettre sous la forme de user stories. C'est une description simple de ce que l'utilisateur a besoin pour savoir les différentes fonctionnalités à développer. J'ai décidé de mettre aussi en place la méthode MoSCoW qui attribue des priorités sur les tâches afin de pouvoir s'attarder sur ce qui est prioritaire. Les niveaux de priorités sont :

- P1 Must
- P2 Should
- P3 Could

Toutes ces stories sont mises dans un product backlog. J'ai fait un planning effectif pour pouvoir comparer mon avancée avec le planning prévisionnel.

3. Décider

Lors de mon projet, j'ai eu plusieurs décisions importantes à prendre. Toutes les décisions que je trouvais importantes et pertinentes, je les mettais dans le journal de bord et j'expliquais pourquoi j'ai choisi de faire comme ça.

4. Réaliser

Une fois les décisions prises, je pouvais commencer à réaliser, soit l'implémentation dans le code, soit la rédaction dans la documentation.

5. Contrôler

tout de suite plusieurs fois. Quand je rajoutais une fonction, je testais également l'autre pour voir s'il y avait pas de régression. Une fois que je finissais d'implémenter un système, je la testais avec des test unitaires et fonctionnels pour voir si tout fonctionnait correctement. (Unity Test Framework)

6. Evaluer

Cette étape je la fais dans mon bilan dans mon journal de bord. Ici on liste les réussites, les difficultés, les améliorations ainsi que les décisions. J'ai aussi mis le resultat de mes tests.

Méthode Agile

Durant l'élaboration de mon projet, je n'ai pas seulement appliqué la méthode en 6 étapes mais également celle de la méthode Agile. Je me suis fixé tous les jours des objectifs dans le journal de bord que je devais atteindre en fin de journée.

Le backlog fait également partie de la méthodologie agile puisque j'ai fait des stories qui correspondent à de petits objectifs à atteindre pour arriver à l'objectif final. Les objectifs sont atteints quand les cas de tests sont fonctionnels et ainsi pouvoir passer au suivant.

Sauvegardes et versionning

Pour versionner mon projet, j'ai utilisé Git. J'ai créé un dépôt sur GitHub pour pouvoir sauvegarder mon code source et le partager avec mon maître d'apprentissage et les experts.

Concernant mon organisation des sauvegardes, j'ai décidé d'adopter la méthode 3-2-1. Cela signifie que je garde 3 copies de mes données, sur 2 supports différents, dont 1 hors site. J'ai donc sauvegardé mon code source sur GitHub, sur un disque dur externe et sur un google drive.

Planification

J'ai fais un planning prévisionnel pour pouvoir m'organiser et savoir ce que je dois faire en découpant les user stories en plusieurs tâches. Pour ces tâches, j'ai décidé de les mettre sous la forme de user stories. C'est une description simple de ce que l'utilisateur a besoin pour savoir les différentes fonctionnalités à développer.

Planning prévisionnel

 Planning prévisionnel

Planning effectif

 Planning effectif

Product backlog

Les user stories sont des descriptions simples de ce que l'utilisateur a besoin pour savoir les différentes fonctionnalités à développer. Ces dernières sont mises dans un product backlog.

les niveaux de priorités sont :

- P1 Must
- P2 Should
- P3 Could

ID	Nom	Description	Priorité
001	Implémentation VR	En tant qu'utilisateur, je veux pouvoir interagir avec le jeu en utilisant un casque VR et ses contrôleurs.	P1
002	Environnement 3D	En tant qu'utilisateur, je veux pouvoir évoluer dans un jardin en 3D de 2x2 mètres. et ne pas pouvoir sortir de la zone de jeu.	P1
003	Main du joueur	En tant qu'utilisateur, je veux voir les contrôleurs dans le jeu sous forme de mains.	P1
004	Arbres et buissons	En tant qu'utilisateur, je veux que le jardin contienne des arbres et des buissons. (Une dizaine)	P1
005	Génération de fruits	En tant qu'utilisateur, je veux que des fruits apparaissent aléatoirement sur des arbres ou des buissons. Les fruits doivent apparaître à une vitesse définie.	P1
006	Ramassage de fruits	En tant qu'utilisateur, je veux pouvoir ramasser des fruits en les ramassant avec les contrôleurs.	P1
007	Disparition des fruits	En tant qu'utilisateur, je veux que les fruits disparaissent après quelques secondes s'ils ne sont pas ramassés	P1
008	Compteur de points	En tant qu'utilisateur, je veux pouvoir ramasser des fruits pour gagner des points. Chaque fruit ramassé donne 1 point.	P1
009	Visualisation des points	En tant qu'utilisateur, je veux voir le nombre de fruits que j'ai ramassé.	P1
010	Compteur de	En tant qu'utilisateur, je veux voir le temps restant pour la	P1

ID	Nom	Description	Priorité
	temps	partie.	
011	Fin de partie	En tant qu'utilisateur, je veux que la partie se termine après 30 secondes.	P1
012	Score final	En tant qu'utilisateur, je veux voir mon score final à la fin de la partie.	P1
013	Rejouer	En tant qu'utilisateur, je veux pouvoir rejouer après avoir vu mon score final.	P1
014	Musique et bruitages	En tant qu'utilisateur, je veux entendre de la musique et des bruitages.	P2
015	Graphismes	En tant qu'utilisateur, je veux que le jeu soit agréable visuellement.	P2
016	Modèles 3D	En tant qu'utilisateur, je veux que les modèles 3D soient de qualité.	P2
017	Post-traitement	En tant qu'utilisateur, je veux que le jeu soit agréable visuellement grâce à un post-traitement.	P3
018	Interface	En tant qu'utilisateur, je veux que l'interface soit intuitive.	P3

Tâches techniques

Les tâches techniques sont des tâches plus précises qui permettent de réaliser les user stories. Elles sont plus techniques et détaillées et ne sont pas destinées au client. Nous avons décidé de les formuler sous la forme de petites tâches issues des user stories.

000 : Préparation du projet

ID	Nom	Description	Priorité
000.1	Création du repository Git	Création du repository Git pour versionner le code	P1
000.2	Création du journal de bord	Création du journal de bord pour suivre l'avancement du projet	P1
000.3	Création de la documentation	Création de la documentation pour expliquer le projet	P1

ID	Nom	Description	Priorité
000.4	Planification	Planification du projet	P1
000.5	Création du projet Unity	Création du projet Unity	P1

001 : Implémentation VR

ID	Nom	Description	Priorité
001.1	Ajout du package XR	Ajout du package XR (Plugin provider : OpenXR)	P1
001.2	Ajout Unity Input System	Ajout du package Unity Input System requis pour le package XR Interaction Toolkit	P1
001.3	Ajout du package XR Interaction Toolkit	Ajout du package XR Interaction Toolkit	P1
001.4	Ajout du système de déplacement	Ajout du système de déplacement pour pouvoir se déplacer dans le jeu	P1

002 : Environnement 3D

ID	Nom	Description	Priorité
002.1	Création du jardin	Création du jardin en 3D de 2x2 mètres	P1
002.2	Ajout des limites	Ajout des limites pour ne pas pouvoir sortir de la zone de jeu	P1
002.3	Ajout des arbres et buissons	Ajout des arbres et des buissons dans le jardin (ce qui vont servir à générer les fruits plus tard)	P1

003 : Main du joueur

ID	Nom	Description	Priorité
001.1	Ajout des contrôleurs	Ajout des contrôleurs pour pouvoir interagir avec le jeu	P1
001.2	Importation des modèles de mains	Importation des modèles de mains pour les contrôleurs	P1

ID	Nom	Description	Priorité
001.3	Ajout du système d'interaction	Ajout du système d'interaction pour pouvoir interagir avec les objets du jeu	P1

004 : Arbres et buissons

ID	Nom	Description	Priorité
004.1	Recherche de modèles 3D	Recherche de modèles 3D d'arbres et de buissons	P1
004.2	Importation des modèles 3D	Importation des modèles 3D d'arbres et de buissons dans le projet	P1
004.3	Placement des arbres et buissons	Placement des arbres et des buissons dans le jardin	P1

005 : Génération de fruits

ID	Nom	Description	Priorité
005.1	Conception du système de fruits	Conception du système de génération de fruits + UML	P1
005.2	Implémentation du système de fruits	Implémentation du système de génération de fruits	P1
005.3	Test du système de fruits	Test du système de génération de fruits	P1

006 : Ramassage de fruits

ID	Nom	Description	Priorité
006.1	Utilisation du système d'interaction et celui des fruits	Utilisation le système d'interaction pour ramasser les fruits	P1
006.2	Test du système de ramassage	Test du système de ramassage des fruits	P1

007 : Disparition des fruits

ID	Nom	Description	Priorité
007.1	Rajout d'un timer sur les fruits	Rajout d'un timer sur les fruits pour les faire disparaître après quelques secondes	P1
007.2	Test du système de disparition	Test du système de disparition des fruits	P1
007.3	Optimisation du système	Optimisation du système de disparition des fruits en utilisant du pooling	P2

008 : Compteur de points

ID	Nom	Description	Priorité
008.1	Création du système de points	Création du système de points pour compter les points	P1
008.2	Test du système de points	Test du système de points	P1

009 : Visualisation des points

ID	Nom	Description	Priorité
009.1	Création de l'interface	Création de l'interface pour afficher les points	P1
009.2	Test de l'interface	Test de l'interface pour afficher les points	P1

010 : Compteur de temps

ID	Nom	Description	Priorité
010.1	Création du timer	Création du timer de 30 secondes pour la partie	P1
010.2	Test du timer	Test du timer de 30 secondes	P1
010.3	Affichage du timer	Rajout de l'affichage du timer à l'interface	P1

011 : Fin de partie

ID	Nom	Description	Priorité
011.1	Fin de partie	Fin de partie après 30 secondes	P1
011.2	Test de la fin de partie	Test de la fin de partie après 30 secondes	P1

012 : Score final

ID	Nom	Description	Priorité
012.1	Affichage du score final	Affichage du score final à la fin de la partie	P1
012.2	Test de l'affichage du score final	Test de l'affichage du score final à la fin de la partie	P1

013 : Rejouer

ID	Nom	Description	Priorité
013.1	Création du système de chargement de scene	Création du système de chargement de scene pour pouvoir rejouer	P1
013.2	Test du système de chargement de scene	Test du système de chargement de scene pour pouvoir rejouer	P1
013.3	Implémentation du système dans l'interface	Implémentation du système de chargement de scene dans l'interface (bouton)	P2
013.3	Optimisation du système	Optimisation du système de chargement de scene pour qu'il soit asynchrone	P2
013.4	Ajout d'un écran de chargement	Ajout d'un écran de chargement pour le chargement de la scene	P3
013.5	Test de l'écran de chargement	Test de l'écran de chargement pour le chargement de la scene	P3

014 : Musique et bruitages

ID	Nom	Description	Priorité
014.1	Recherche de musiques et bruitages	Recherche de musiques et bruitages pour le jeu	P2
014.2	Importation des musiques et bruitages	Importation des musiques et bruitages dans le projet	P2
014.3	Ajout des musiques et bruitages	Ajout des musiques et bruitages dans le jeu	P2

015 : Graphismes

ID	Nom	Description	Priorité
015.1	Recherche d'une identité visuelle	Recherche d'une identité visuelle pour le jeu	P2
015.2	Elaboration de la palette de couleurs	Elaboration de la palette de couleurs pour le jeu	P2

016 : Modèles 3D

ID	Nom	Description	Priorité
016.1	Recherche de modèles 3D	Recherche de modèles 3D pour le jeu	P2
016.2	Importation des modèles 3D	Importation des modèles 3D dans le projet	P2
016.3	Ajout des modèles 3D	Ajout des modèles 3D dans le jeu	P2

017 : Post-traitement

ID	Nom	Description	Priorité
017.1	Ajout d'un post-traitement	Ajout d'un post-traitement pour améliorer les graphismes	P3

018 : Interface

ID	Nom	Description	Priorité
018.1	Conception de l'interface	Conception de l'interface pour qu'elle soit intuitive et en harmonie avec l'idée visuelle	P3
018.2	Implémentation de l'interface	Implémentation de l'interface dans le jeu	P3

Analyse organique

Pour faciliter la partie conception ainsi que la partie implémentation, le projet a été divisé en plusieurs systèmes. Chaque système a une responsabilité bien définie et est plus ou moins indépendant des autres systèmes. Cela permet de faciliter la maintenance et l'évolution du projet.

Informations générales

Pour faciliter l'organisation du code, chaque système possède une définition d'assembly.

Nom	Description	Chemin (racine du projet unity)
Scripts	Scripts généraux	Assets/Scripts/Scripts.asmdef
FruitSystem	Système de gestion des fruits	Assets/Scripts/FruitSystem/FruitSystem.asmdef
Inputs	Système de gestion des inputs	Assets/Scripts/Inputs/Inputs.asmdef
UISystem	Système de gestion de l'interface utilisateur	Assets/Scripts/UI/UISystem.asmdef
Audio	Système de gestion de l'audio	Assets/Scripts/Audio/Audio.asmdef

Nom	Description	Chemin (racine du projet unity)
GameManagement	Système de gestion de la partie	Assets/Scripts/GameManagement/GameManagement.asmdef

Dans la même optique, l'utilisation de namespaces a été privilégiée pour une meilleure organisation du code.

Remarques :

Gestions de la réalité virtuelle

La gestion de la réalité virtuelle est un des points clés du projet. Elle permet au joueur d'interagir avec le jeu en utilisant un casque de réalité virtuelle et ses contrôleurs. Pour cela, nous avons utilisé le package XR d'Unity qui permet de gérer la réalité virtuelle de manière simple et efficace. Nous avons également utilisé le package XR Interaction Toolkit qui permet de gérer l'interaction avec les objets du jeu. Pour gagner du temps, nous avons utilisé le sample de l'XR Interaction Toolkit pour la gestion des contrôleurs. Ce dernier nous a permis d'avoir tout de suite les `InputsActions` et les interactions de base (grab, select, etc).

Mains du joueur

Pour les mains du joueur, nous avons utilisé les modèles de mains donnés dans une série de tutoriels de Unity. Ces modèles sont des modèles de mains de base qui sont déjà animés. Nous avons simplement suivi le tutoriel pour les importer dans le projet et les utiliser.

Référence de la vidéo : [How to Make a VR Game in Unity 2022 - PART 2 - INPUT and HAND PRESENCE](#)

La seule classe que nous avons dû implémenter est lui aussi donné dans le tutoriel. Il s'agit de la classe `HandController` qui permet de faire le lien entre les contrôleurs et l'animateur des mains.

Deplacement du joueur

Les déplacements du joueur sont très simples étant donné que le joueur ne peut pas se déplacer dans l'environnement avec les contrôleurs. Il ne peut que se déplacer dans un rayon de 2 mètres autour de lui en marchant dans la réalité. Pour cela, nous avons utilisé les composants `LocomotionSystem`, `ContinuousMoveProvider` et `CharacterControllerDriver` du package XR Interaction Toolkit.

Utilisation dans Unity

XR Plugin Management

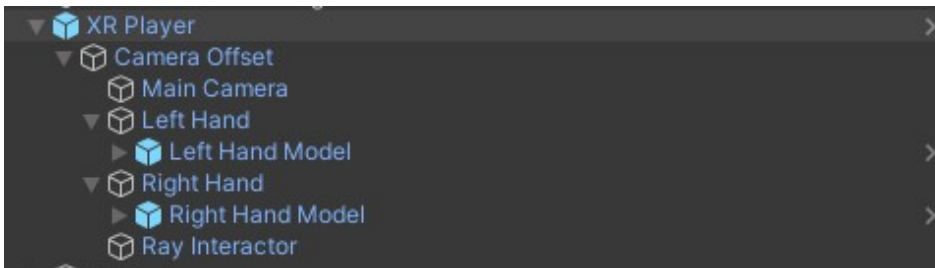
Pour utiliser la réalité virtuelle dans Unity, il faut ajouter le package XR dans le projet. Pour cela, il faut aller dans le menu `Window -> Package Manager` et chercher le package `XR Plugin Management`. Il faut ensuite l'installer. Une fois le package installé, il faut aller dans `Edit -> Project Settings -> XR Plugin Management` et activer le plugin `OpenXR`.

XR Interaction Toolkit

Pour utiliser le package `XR Interaction Toolkit`, il faut ajouter le package dans le projet. Pour cela, il faut aller dans le menu `Window -> Package Manager` et cliquer sur le `+` en haut à gauche `> Add package from name` et entrer `com.unity.xr.interaction.toolkit`. Il faut ensuite l'installer. Pour installer le sample `Started Assets`, il faut aller dans `Package Manager > XR Interaction Toolkit > Samples > Started Assets > Import`.

Utilisation dans la scène

L'utilisation de la réalité virtuelle dans la scène est très simple. Voici la hiérarchie du joueur dans la scène :



XR Player

C'est le `GameObject` qui représente le joueur dans la scène.

Voici la vue de l'inspecteur du `XR Player` :

XR Player

Static

Tag Untagged Layer Player

Prefab XR Player

Overrides

Select

Open

Transform

Position

X 0 Y 0 Z 0

Rotation

X 0 Y 0 Z 0

Scale

X 1 Y 1 Z 1

XR Origin

Script

XR Origin

Origin Base GameObject

XR Player

Camera Floor Offset GameObject

Camera Offset

Camera

Main Camera (Camera)

Tracking Origin Mode

Floor

Input Action Manager

Script

InputActionManager

Action Assets

1

Element 0

XRI Default Input Actions (Input Action Asset)

Locomotion System

Script

LocomotionSystem

Timeout

10

XR Origin

XR Player (XR Origin)

Continuous Move Provider (Action-based)

Script

ActionBasedContinuousMoveProvider

System

XR Player (Locomotion System)

Move Speed

1

Enable Strafe

☒

Enable Fly

☐

Use Gravity

☒

Gravity Application Mode

Immediately

Forward Source

Main Camera (Transform)

Left Hand Move Action

Use Reference

☐

Action

Right Hand Move Action

Use Reference

☐

Action

Character Controller Driver

Script

CharacterControllerDriver

Locomotion Provider

XR Player (Action Based Continuous Move Provider)

Min Height

0

Max Height

Infinity

Character Controller

Voici la liste des composants du XR Player :

- `XROrigin` : Composant qui permet de définir l'origine du joueur dans la scène.
- `InputActionManager` : Composant qui permet de gérer les actions des contrôleurs.
- `LocomotionSystem` : Composant qui permet de gérer le déplacement du joueur.
- `ContinuousMoveProvider` : Composant qui permet de gérer le déplacement du joueur (aucun référence au Move Action car le joueur ne peut pas se déplacer avec les contrôleurs).
- `CharacterControllerDriver` : Composant qui permet de gérer le déplacement du joueur.
- `CharacterController` : Composant qui permet de gérer le déplacement du joueur (n'est pas propre à la VR).

LeftHand et RightHand

Les mains du joueur sont des GameObjects enfants du XR Player. Voici la vue de l'inspector d'une main :

Inspector

Left Hand

Static

Tag Untagged Layer Player

Transform

Position

X 0Y 0Z 0

Rotation

X 0Y 0Z 0

Scale

X 1Y 1Z 1

XR Controller (Device-based)

ScriptXRController

Tracking

Update Tracking TypeUpdate And Before Render

Enable Input Tracking

Input

Enable Input Actions

Pose ProviderNone (Base Pose Provider)

Controller NodeLeft Hand

Select UsageGrip

Activate UsageTrigger

UI Press UsageTrigger

Axis To Press Threshold0.1

Rotate Object LeftPrimary Axis 2D Left

Rotate Object RightPrimary Axis 2D Right

Move Object InPrimary Axis 2D Up

Move Object OutPrimary Axis 2D Down

Directional Anchor RotationPrimary Axis 2D

Model

Model PrefabNone (Transform)

Model ParentNone (Transform)

ModelNone (Transform)

Animate Model

XR Direct Interactor

ScriptXRDirectInteractor

Interaction ManagerXR Interaction Manager (XR Interaction Manager)

Interaction Layer MaskEverything

Attach TransformNone (Transform)

Disable Visuals When Blocked In Group

Improve Accuracy With Sphere Collider

Select Action TriggerState Change

Keep Selected Target Valid

Hide Controller On Select

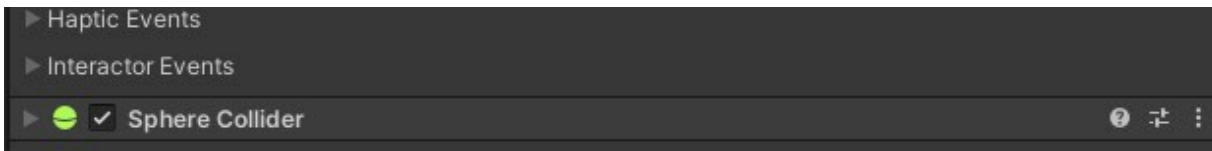
Allow Hovered Activate

Target Priority ModeNone

Starting Selected InteractableNone (XR Base Interactable)

Interactor Filters

Audio Events



Voici la liste des composants d'une main :

- `XRController` : Composant qui permet de gérer le contrôleur.
- `XRDirectInteractor` : Composant qui permet de gérer l'interaction avec les objets (les attraper, les lancer, etc).
- `SphereCollider` : Composant qui permet de gérer la zone de détection des objets. (mis en mode trigger pour ne pas bloquer les objets).

En plus de ces composants, il y a un `GameObject` enfant de la main qui contient le modèle de la main. Ce `GameObject` est animé par l'Animator de la main.

Il possède également un `HandController` ([HandController](#)) qui permet de faire le lien entre les contrôleurs et l'Animator de la main.

Environnement 3D

L'environnement 3D est un jardin clos de 2m x 2m. Il contient des arbres et des buissons qui servent à générer les fruits, des murs pour délimiter la zone de jeu et un sol pour marcher. La scène est composée de plusieurs `GameObjects` qui sont organisés de manière à ce que le joueur puisse se déplacer librement dans la zone de jeu.



Un panier est également présent dans la scène pour que le joueur puisse y mettre les fruits qu'il a ramassé pour gagner des points. Ce dernier est en hauteur pour que le joueur puisse y mettre les fruits facilement qu'il mesure 60cm ou 1m90. Il est légèrement éclairé pour le mettre en valeur et soit perçu comme un élément important.

Une radio est également présente dans la scène pour que le joueur puisse entendre de la musique. Elle est placée à côté du panier et est également éclairée (moins que le panier pour ne pas trop attirer l'attention).

Etant donné que ce projet va être utilisé dans le cadre des portes ouvertes du CFPT, les joueurs devront comprendre rapidement comment jouer. Pour cela, un texte est affiché sur un mur pour expliquer qu'il faut ramasser les fruits et les mettre dans le panier pour gagner des points.

Des éléments de décorations sont également présents dans la scène pour rendre le jardin plus vivant. Il y a des fleurs, des cailloux, des champignons, des fougères, etc.



L'ambiance de la scène est très importante pour que le joueur se sente bien dans le jeu. Une ambiance de coucher de soleil faisant contraster ses couleurs chaudes avec les couleurs vives des éléments du jardin permet de donner une ambiance chaleureuse ainsi que de mettre en valeur les fruits et autres éléments clés du jeu.



Gestion des parties

Le système de gestion de partie est relativement simple. C'est un singleton qui utilise un pattern d'observer pour notifier les autres systèmes des différents événements de la partie. Ce système est composé de plusieurs classes qui gèrent les différents aspects de la partie (score, timer, statistiques, etc).

Le déroulement d'une partie est le suivant :

1. Le joueur appuie sur le bouton de démarrage de la partie.
2. Un compte à rebours de 3 secondes est lancé pour laisser le temps au joueur de se préparer.
3. La partie commence et les fruits commencent à apparaître.
4. Le joueur doit ramasser les fruits et les mettre dans le panier pour gagner des points.
5. Après 30 secondes, la partie se termine et le score final est affiché.
6. Le joueur peut rejouer en appuyant sur un bouton.



Classes du système de gestion de partie

GameManager

Le `GameManager` est la classe principale du système de gestion de partie. C'est un singleton qui centralise toutes les opérations sur la partie. Utilisant un pattern d'observer, il notifie les autres systèmes de l'état de la partie.

GameOption

La structure `GameOption` est une structure qui contient les options de la partie. Elle est utilisée par le `GameManager` pour modifier les options de la partie dans l'éditeur Unity et en cours d'exécution (bien que cela ne soit pas nécessaire car aucun menu d'options n'est implémenté).

Elle contient les champs suivants :

- `public float gameDuration` : La durée de la partie en secondes.
- `public ushort spawnerRate` : Le nombre d'apparition de chaque type de fruits par seconde.
- `public uint countdownDuration` : La durée du compte à rebours du début de partie en secondes.

GameScore

La classe `GameScore` est une classe qui contient le score du joueur. Elle est utilisée par le `GameManager` pour gérer le score du joueur.

GameTimer

La classe `GameTimer` contient le timer de la partie. Elle utilise une coroutine pour le timer. Elle contient 3 Action (données en paramètre du constructeur) qui sont appelées à différents moments du timer. (début, 80% du timer, fin). Ces actions permettent d'abstraire le comportement sans avoir de dépendances entre les classes.

GameStats

La classe `GameStats` est une classe qui s'occupe de stocker les statistiques de la partie (pour le moment, uniquement les fruits ramassés). Elle est utilisée par le `GameManager`.

Gestion des fruits

Le système de gestion des fruits est un des systèmes les plus importants du projet. Il est responsable de la génération des fruits, de leur apparition, de leur disparition, de leur ramassage et de leur comptage.

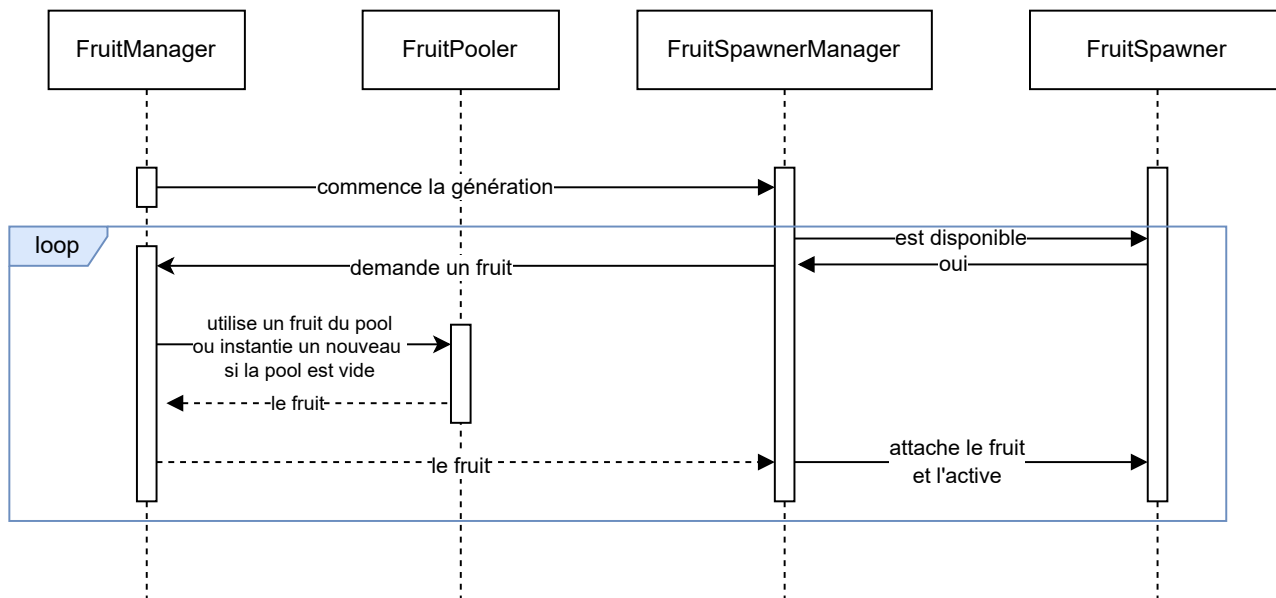
Avant d'aborder les détails du système, il est important de comprendre comment les fruits sont générés. Premièrement, il existe plusieurs types de fruits (pommes, fraises, myrtilles). Chaque type de fruit a des caractéristiques différentes (points donnés, vitesse d'apparition, etc). Deuxièmement, les fruits apparaissent sur des arbres ou des buissons. Les pommes apparaissent sur les arbres, les fraises et les myrtilles apparaissent sur les buissons. Ils apparaissent à une vitesse définie :

$$V_a = \frac{R}{p}$$

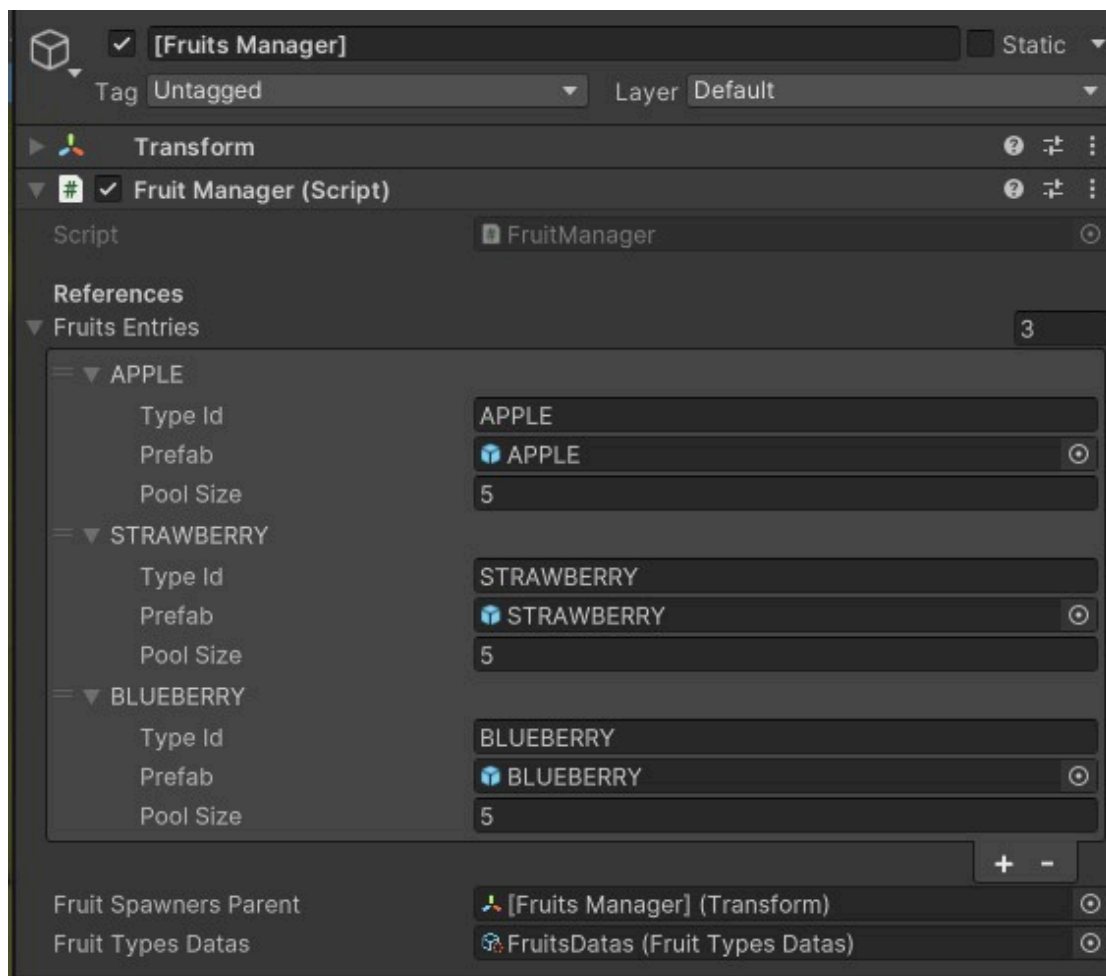
où :

- V_a est la vitesse d'apparition des fruits. [fruits/s]
- p (points) est le nombre de points donnés par le fruit.
- R (rate) est le nombre de fruits qui vont apparaître par seconde (en même temps).

Chaque fruit a un temps de vie configurable. Si le joueur ne les ramasse pas avant la fin de leur temps de vie, ils disparaissent. Quand le joueur ramasse un fruit, sa durée de vie est stoppée et ne disparaît pas. C'est seulement quand le joueur lâche le fruit qu'il reprend sa durée de vie (qui est remise à zéro). Une fois que le fruit est mis dans le panier des points sont ajoutés au score du joueur.

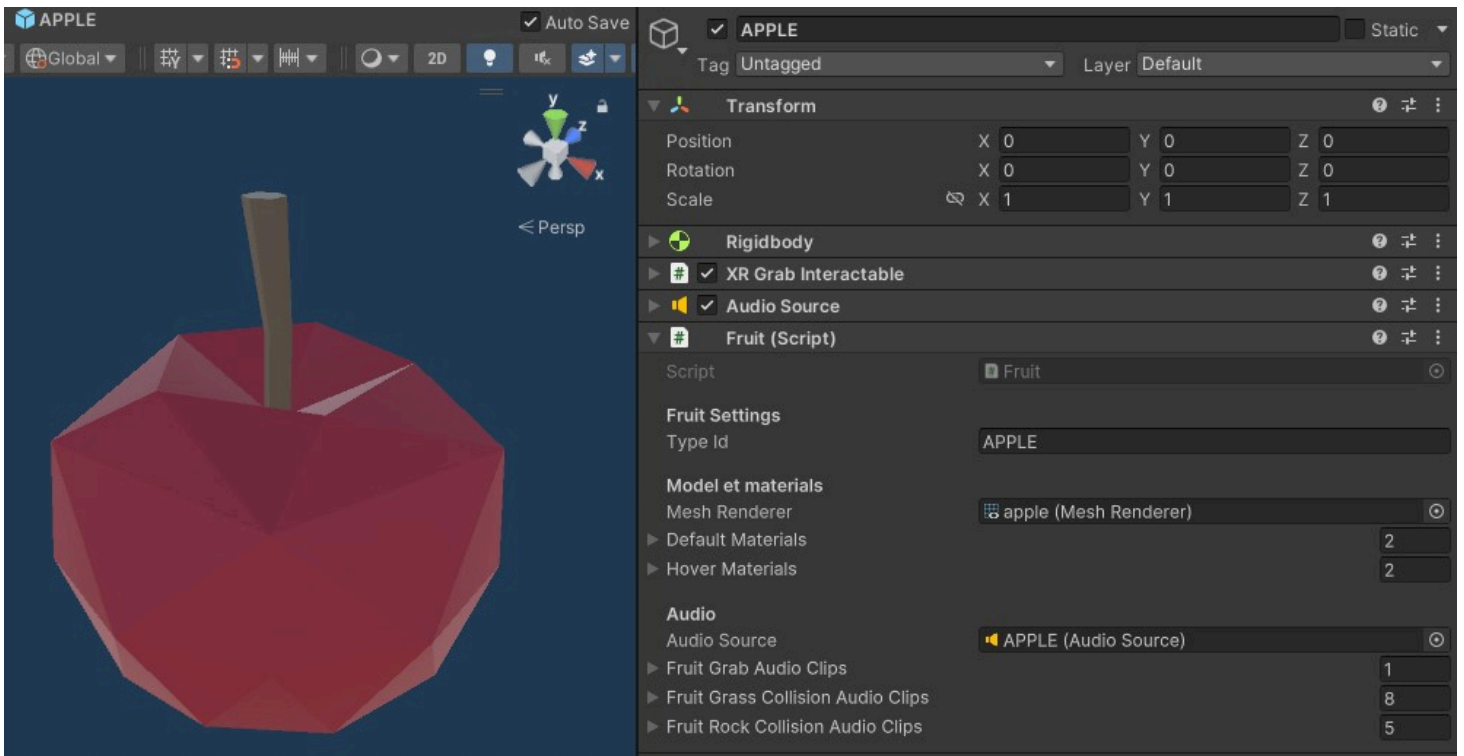


Globalement, le système est composé d'une classe **FruitManager** qui centralise toutes les opérations sur les fruits. Ce dernier utilise un **FruitPooler** pour gérer les fruits en pool. Il permet de réutiliser les fruits déjà instanciés pour éviter de les créer et de les détruire à chaque fois, cela permet un gain de performance non négligeable.

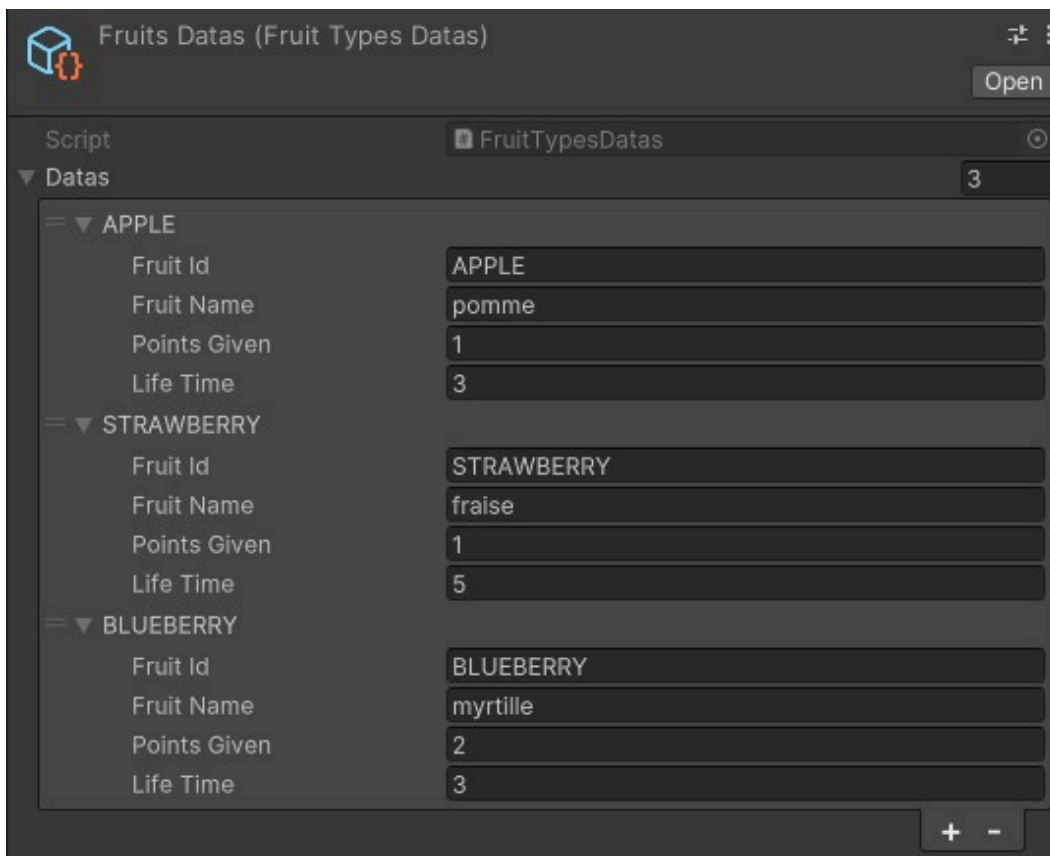


La classe [Fruit](#) représente un fruit dans le jeu. Il contient les informations sur le fruit (type, points, etc) et les méthodes pour le ramasser et le détruire.

Dans l'éditeur Unity, un fruit est représenté par un prefab qui contient un rigidbody, un composant XR Grab Interactable, une source audio et la classe Fruit.



Pour regrouper et donner un accès facile aux données de chaque fruit, une structure [FruitTypeData](#) est utilisée. Elle contient les informations sur le fruit (type, points, etc). Cette dernière est utilisée dans [FruitTypesDatas](#) une classe héritant de ScriptableObject qui permet de stocker les données des fruits dans l'éditeur Unity.



Un [FruitSpawnerManager](#) est utilisé pour gérer les [FruitSpawner](#). C'est sur ces derniers que la position des fruits est définie. Ils sont placés sur les arbres et les buissons pour que les fruits apparaissent à ces endroits.



les points rouge représentent les spawners de pommes, les verts les fraises et les bleus les myrtilles.

Classes du système de fruit

FruitManager

Le `FruitManager` est la classe principale du système de gestion des fruits. Elle est responsable a haut niveau de toutes les opérations sur les fruits. Elle implémente un pattern singleton pour donner un accès facile aux autres classes du système ainsi qu'aux autres systèmes.

Cette classe utilise un `FruitPooler` pour gérer les fruits en pool. Un `FruitSpawnerManager` est également utilisé pour gérer les `FruitSpawner`.

Elle contient une référence au `FruitTypesDatas` qui contient les données des fruits.

Grâce à la methode `public static GetFruitTypeData(string fruitId)` il est possible de récupérer les

données d'un fruit en donnant son id. Cela permet de facilement accéder a ces données depuis d'autres systèmes (par exemple, pour afficher le nom des fruits ramassés dans l'interface de fin de partie).

FruitPooler

Le `FruitPooler` est une classe qui gère les fruits en pool. Elle permet de réutiliser les fruits déjà instanciés pour éviter de les créer et de les détruire à chaque fois, cela permet un gain de performance non négligeable.

Son fonctionnement est simple. Au démarrage de la partie, elle instancie un nombre de fruits défini dans l'éditeur Unity. Ces fruits sont ensuite désactivés et organisés dans un dictionnaire qui contient l'id du type de fruit et une queue de fruits. Quand un fruit est ramassé, il est désactivé et remis dans la queue. Quand un fruit doit apparaître, il est récupéré de la queue et activé. Si la queue est vide, un nouveau fruit est instancié.

Afin de minimiser un maximum les dépendances entre les classes, l'opération d'instanciation des fruits est passée en paramètre du constructeur de la classe sous la forme d'une

```
Func<Func<ulong, Fruit>, Fruit> .
```

La liste des données nécessaires pour le pooling des fruits est passée en paramètre du constructeur de la classe sous la forme d'un `FruitPoolData[]` .

FruitPoolData

La structure `FruitPoolData` est une structure qui contient les données nécessaires pour le pooling des fruits. Elle est utilisée par le [FruitPooler](#) pour instancier les fruits.

Elle contient les champs suivants :

- `string typeId` : l'identifiant du type de fruit.
- `GameObject prefab` : le prefab du fruit.
- `ushort poolSize` : la taille du pool.

Fruit

Un `Fruit` représente un fruit dans le jeu. Cette classe hérite de `MonoBehaviour` . Ce composant permet de gérer les interactions du joueur avec le fruit (ramassage), gérer sa durée de vie, jouer les différents sons du fruit (apparition, ramassage, collision).

Un fruit peut se trouver dans 3 états différents, représentés par l'énumération [FruitState](#).

Quand un fruit apparaît, il est attaché à un [FruitSpawner](#) - sa position est définie par le spawner et la gravité du fruit est désactivée(état `Grabbed`). Si le joueur ne le ramasse pas avant la fin de sa durée de

vie, il disparaît et est mis dans l'état `Innactive` .

Quand le joueur le ramasse, la gravité est activée et le fruit suit le contrôleur du joueur. Si le joueur le lâche, le fruit est remis dans l'état `Neutral` .

FruitState

L'énumération `FruitState` représente l'état d'un fruit. Un fruit peut se trouver dans 3 états différents :

- `Innactive` : Le fruit est désactivé.
- `Attached` : Le fruit est attaché à un [FruitSpawner](#).
- `Neutral` : Le fruit est actif et peut être ramassé par le joueur. (la gravité est activée).
- `Grabbed` : Le fruit est ramassé par le joueur et suit le contrôleur.

FruitTypeData

La structure `FruitTypeData` est une structure qui contient les données d'un type de fruit. Elle est dans un [FruitTypesDatas](#) qui contient les données de tous les types de fruits.

Elle contient les champs suivants :

- `fruitId` : l'identifiant du fruit.
- `fruitName` : le nom du fruit.
- `points` : le nombre de points donné par le fruit.
- `lifeTime` : la durée de vie du fruit.

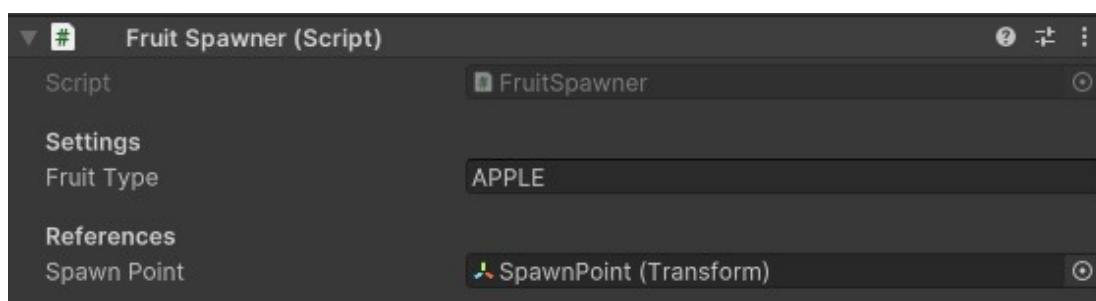
FruitTypesDatas

La classe `FruitTypesDatas` est une classe héritant de `ScriptableObject` qui permet de stocker les données des fruits dans l'éditeur Unity (dans un fichier `.asset`). Elle contient une liste de [FruitTypeData](#) qui contient les données de chaque type de fruit.

FruitSpawner

Un `FruitSpawner` est une classe qui hérite de `MonoBehaviour` et est attachée à un `GameObject`.

Les `FruitSpawner` permettent de définir la position et la rotation sur laquelle les fruits apparaissent. Ils sont placés sur les arbres et les buissons pour que les fruits apparaissent à ces endroits.



FruitSpawnerManager

Le `FruitSpawnerManager` est une classe qui gère les [FruitSpawner](#). Elle gère le spawn des fruits en utilisant des coroutines récursives.

Une liste de [FruitSpawner](#) est passée en paramètre du constructeur de la classe. Cette liste est utilisée pour gérer les spawners.

Encore une fois dans le but de minimiser les dépendances entre les classes, l'opération d'instantiation des fruits est passée en paramètre du constructeur de la classe sous la forme d'une `Func<string, Fruit>`. Pour récupérer tous les fruits une fonction de type `Func<List<Fruit>>` est passée aussi en paramètre du constructeur.

Basket

La classe `Basket` est une classe qui gère le panier. Elle hérite de `MonoBehaviour` et est attachée à un `GameObject` dans la scène. Ce composant est responsable de gérer les fruits qui sont mis dans le panier et de notifier le [GameManager](#) des points gagnés.

Cela est réalisé en utilisant un box collider qui est un trigger. Quand un fruit entre en collision avec ce collider, la méthode `OnTriggerEnter` est appelée. Cette méthode vérifie si le fruit est un fruit et si oui, elle joue le son de ramassage, ajoute les points au score et désactive le fruit.

Interface utilisateur

Implémentation

Généralités concernant l'implémentation

- [Unity 2022.3.12f1](#)
- [.NET Standard 2.1](#)
- [C# 9.0](#)
- compilateur C# : [Roslyn](#)

Librairies et outils externes

- [DoTween](#)
- [Unity Test Framework](#)
- [XR Plugin Management](#)

- [XR Interaction Toolkit](#)

Analyse des fonctionnalités majeures

Sécurité

Etant donné que le projet est un jeu vidéo solo, nous avons décidé de ne pas mettre la priorité sur la sécurité. Si le joueur veut tricher, il peut le faire. Cependant, nous avons quand même mis en place un obfuscateur pour éviter le reverse engineering (surtout car `c#` est un langage facile à décompiler).

Obfuscation

Nous avons utilisé l'obfuscateur [Obfuscator Free](#) de GuardingPearSoftware pour protéger notre code source.

Plan de test

Périmètre

Le plan de test a pour but de valider les fonctionnalités principales du projet. Certains test sont effectués manuellement, d'autres sont automatisés. Les tests manuels sont effectués par le développeur pour vérifier le bon fonctionnement des fonctionnalités. Les tests automatisés sont effectués par le framework de test de Unity pour vérifier le bon fonctionnement des fonctionnalités de manière automatique.

UnityTestFramework est un framework de test intégré à Unity qui permet de tester les fonctionnalités de l'application. Il est divisé en deux parties : les tests live et les tests en mode édition. Les tests live sont des tests qui sont exécutés en même temps que l'application. Les tests en mode édition sont des tests qui sont exécutés sans que l'application ne soit en cours d'exécution.

Cas de test

Tests manuels

L'identifiant (ID) est formé de la lettre M (pour manuel) suivi d'un numéro.

ID	Description	Préconditions	Actions	Résultat attendu
M1	Le projet se lance et il fonctionne bien avec le	Casque connecté et pc fonctionnel	Lancer le projet	Le projet se lance et fonctionne bien

ID	Description	Préconditions	Actions	Résultat attendu
	casque de réalité virtuel			
M2	Le joueur se trouve dans un jardin virtuel clos de 2x2m	Le projet est lancé	Regarder autour de soi	Le joueur se trouve dans un jardin virtuel clos de 2x2m
M3	Le joueur peut se déplacer dans le monde virtuel	Le projet est lancé	se déplacer dans la vrai vie	Le joueur se déplace dans le monde virtuel
M4	Le joueur ne peut pas sortir du jardin virtuel	Le joueur est dans le jardin virtuel	Essayer de sortir du jardin	Le joueur ne peut pas sortir du jardin virtuel
M5	Un menu s'affiche et suis le joueur	Aucune partie n'est en cours	regarder devant soi	Un menu s'affiche et suis le joueur
M6	Le joueur peut lancer une partie depuis l'interface en utilisant le bouton "Jouer"	Le menu est affiché	Appuyer sur le bouton "Jouer"	Une partie se lance
M7	Un HUD s'affiche en haut de l'écran (dans le casque)	Une partie est en cours	Regarder en haut de l'écran	Un HUD s'affiche en haut de l'écran
M8	Un compteur de 30 secondes s'affiche dans le HUD	Une partie est en cours	Regarder le compteur	Un compteur de 30 secondes s'affiche dans le HUD
M9	Un compteur de score s'affiche dans le HUD	Une partie est en cours	Regarder le compteur	Un compteur de score s'affiche dans le HUD
M10	Des fruits apparaissent aléatoirement dans le jardin virtuel	Une partie est en cours	Regarder autour de soi	Des fruits apparaissent aléatoirement dans le jardin virtuel
M11	Les fruits disparaissent après 5 secondes	Une partie est en cours	Attendre 5 secondes	Les fruits disparaissent après 5 secondes

ID	Description	Préconditions	Actions	Résultat attendu
M12	Le joueur peut interagir avec les fruits en les attrapant	Une partie est en cours	Attraper un fruit	Le joueur attrape un fruit
M13	Le joueur peut mettre un fruit dans un panier pour gagner des points	Une partie est en cours	Mettre un fruit dans un panier	Le joueur gagne des points
M14	Un fruit mis dans un panier disparaît	Une partie est en cours	Mettre un fruit dans un panier	Le fruit disparaît
M15	Une fois le compteur à 0, la partie se termine	Une partie est en cours	Attendre que le compteur soit à 0	La partie se termine
M16	Une fois une partie terminée, un écran de fin de partie s'affiche	Une partie est terminée	Regarder l'écran de fin de partie	Un écran de fin de partie s'affiche
M17	Le score du joueur s'affiche à la fin de la partie	Une partie est terminée	Regarder le score	Le score du joueur s'affiche à la fin de la partie
M18	Un bouton "Rejouer" s'affiche à la fin de la partie et permet de lancer une nouvelle partie	Une partie est terminée	Appuyer sur le bouton "Rejouer"	Une nouvelle partie se lance
M19	Une musique de fond est jouée pendant la partie	Une partie est en cours	Ecouter la musique	Une musique de fond est jouée pendant la partie
M20	Un son est joué quand le joueur attrape un fruit	Une partie est en cours	Attraper un fruit	Un son est joué quand le joueur attrape un fruit
M21	Un son est joué quand le joueur met un fruit dans un panier	Une partie est en cours	Mettre un fruit dans un panier	Un son est joué quand le joueur met un fruit dans un panier

ID	Description	Préconditions	Actions	Résultat attendu
M22	Un son est joué quand la partie se termine bientôt (80%)	Une partie est en cours	Attendre que la partie se termine bientôt	Un son est joué quand la partie se termine bientôt
M23	Un son est joué quand un fruit entre en collision avec le sol	Une partie est en cours	Laisser un fruit tomber	Un son est joué quand un fruit entre en collision avec le sol
M24	La couleur du fruit s'éclaircit quand le joueur peut l'attraper	Une partie est en cours	Approcher la main du fruit	La couleur du fruit s'éclaircit quand le joueur peut l'attraper

Tests automatisés

Les tests automatisés visent uniquement le système de gestion des fruits. En effet, c'est le système le plus complexe et le plus critique du projet. Les tests automatisés sont effectués par le framework de test de Unity. Voici les cas de test automatisés :

L'identifiant (ID) est formé de la lettre A (pour automatisé) suivi d'un numéro.

ID	Description	Résultat attendu	détails	fichier
A1	Un fruit avec un type existant doit être instancié correctement par un FruitPooler	Le fruit est instancié correctement	le fruit est instancié, le type est correct, l'id du fruit est correct, l'attributeur d'id est incrémenté, le fruit est ajouté à la liste des fruits, le fruit est ajouté a la hierrachie en tant qu'enfant du parent definis	FruitTest.cs
A2	Une exception doit être levée si on essaie d'instancier un fruit avec un type inexistant	Une exception est levée	le fruit n'est pas instancié, une exception <code>FruitTypeIdDoesNotExistException</code> est levée	FruitTest.cs

[illegible]

Conclusion

Difficultés rencontrées

Pendant la réalisation de ce projet, plusieurs difficultés ont été rencontrées. Voici quelques exemples de difficultés rencontrées :

Variantes de solutions et choix

Pour résoudre ces difficultés, plusieurs solutions ont été envisagées. Voici quelques exemples de variantes de solutions et de choix effectués :

Améliorations possibles

Pour améliorer le projet, voici quelques pistes d'améliorations possibles :

Bilan personnel

Ce projet m'a permis de mettre en pratique les compétences acquises pendant ma formation. J'ai pu approfondir mes connaissances en C# et en Unity. J'ai également appris à travailler de manière autonome et à gérer mon temps efficacement. Ce projet m'a permis de développer mes compétences en matière de conception et d'implémentation de jeux vidéo en réalité virtuelle.

Remerciements

Je tiens à remercier M. J. Aliprendi pour son soutien et ses conseils tout au long de ce projet. Je tiens également à remercier Mickaël Strazzeri et Yvan Poulin pour leur expertise et leurs retours constructifs.

Références

...

Annexe

Utilisation de chatGPT

Convention doublons documentation

prompt

Dans le cadre d'un projet informatique je dois rédiger une documentation technique.
J'aimerais décrire chaque champ des classes de mon projet en détail.
Pour cela j'utilise un sous-titre ##### Champs pour chaque classe.
Le problème c'est que mon extension vs code m'indique un warning :
"no-duplicate-heading: Multiple headings with the same content"

Quelles sont les conventions/les normes sur ces points ? Comment renommer ces sous-titres pour év:

réponse

L'avertissement que vous recevez indique qu'il y a plusieurs sous-titres avec le même contenu, ce

Nom unique pour chaque champ : Plutôt que d'utiliser simplement "Champs" pour chaque classe, essayez d'utiliser un nom unique pour chaque champ.
Par exemple, si vous avez une classe "Utilisateur", vous pourriez avoir des sous-titres comme "Champs de la classe Utilisateur".
Utilisation des noms de champs : Une autre approche consiste à utiliser les noms réels des champs au lieu de "Champs".
Par exemple, pour la classe "Utilisateur", vous pourriez avoir des sous-titres comme "Nom", "Âge", "Adresse", etc.
Préciser la classe dans le sous-titre : Si vous avez plusieurs classes avec des champs similaires, précisez la classe dans le sous-titre.
Par exemple, pour les sous-titres de la classe "Client" et de la classe "Fournisseur", vous pourriez avoir "Champs de la classe Client" et "Champs de la classe Fournisseur".
Utilisation de la syntaxe du langage : Si votre documentation est destinée à des développeurs ou à des utilisateurs techniques, vous pouvez utiliser la syntaxe du langage pour les sous-titres.
Par exemple, si vous utilisez Java, vous pourriez avoir des sous-titres comme "Attributs de la classe Client" ou "Méthodes de la classe Fournisseur".
Groupement des champs par catégorie : Si vous avez de nombreux champs dans une classe, vous pouvez les regrouper par catégorie.
Par exemple, pour une classe "Produit", vous pourriez avoir des sous-titres comme "Informations de base", "Caractéristiques techniques", etc.
En suivant l'une de ces approches, vous pouvez rendre votre documentation plus claire et éviter les doublons.
En général, la plupart des documentations techniques suivent ces conventions pour organiser et structurer le contenu.

Code source

...